

SHL Assessment Recommendation Chatbot

An AI-powered conversational chatbot that recommends the most relevant SHL assessments based on hiring requirements. The application combines semantic search using FAISS, Google Gemini, FastAPI, and Streamlit to provide accurate, context-aware recommendations from the SHL product catalog.

Features

- AI-powered SHL assessment recommendations
 - Semantic search using FAISS vector database
 - Google Gemini for natural language responses
 - Intent detection (Greeting, Recommendation, Clarification, Comparison, Refinement)
 - Multi-turn conversation support
 - FastAPI REST API backend
 - Streamlit chat interface
 - SHL product catalog search
 - Recommendation with assessment links
-

Tech Stack

- Python 3.10+
 - FastAPI
 - Streamlit
 - Google Gemini API
 - Sentence Transformers
 - FAISS
 - NumPy
 - Pickle
 - Hugging Face Transformers
-

Project Structure

```
SHL chatbot/  
|  
├─ app/  
|   ├── main.py  
|   ├── agent.py  
|   ├── retriever.py  
|   ├── gemini.py  
|   └── prompts.py
```

```
|   |— intent.py
|
|— vectorstore/
|   |— build_vector_db.py
|   |— index.faiss
|   |— metadata.pkl
|
|— streamlit_app.py
|— requirements.txt
|— .env.example
|— README.md
```

Installation

Clone the repository:

```
git clone https://github.com/yourusername/shl-chatbot.git
cd shl-chatbot
```

Create a virtual environment:

```
python -m venv venv
```

Activate the virtual environment:

Windows

```
venv\Scripts\activate
```

Linux / macOS

```
source venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Environment Variables

Create a `.env` file in the project root.

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

Build Vector Database

Run:

```
python vectorstore/build_vector_db.py
```

This generates:

- `index.faiss`
 - `metadata.pkl`
-

Run FastAPI

```
uvicorn app.main:app --reload
```

API Documentation:

```
http://127.0.0.1:8000/docs
```

Health Check:

```
http://127.0.0.1:8000/health
```

Run Streamlit

Open a new terminal and run:

```
streamlit run streamlit_app.py
```

API Endpoint

POST /chat

Example Request

```
{
  "messages": [
    {
      "role": "user",
      "content": "I am hiring a Java Developer with 4 years experience."
    }
  ]
}
```

Example Response

```
{
  "reply": "...",
  "recommendations": [
    {
      "name": "Java 8 (New)",
      "url": "https://www.shl.com/products/product-catalog/view/java-8-new/",
      "test_type": "Knowledge & Skills"
    }
  ],
  "end_of_conversation": false
}
```

Supported Intents

- Greeting
- Recommendation
- Clarification
- Compare Assessments
- Recommendation Refinement
- Off-topic Detection
- Prompt Injection Detection

Workflow

1. User enters hiring requirements.
 2. Intent is detected.
 3. Relevant SHL assessments are retrieved using FAISS.
 4. Gemini generates a natural language response.
 5. FastAPI returns structured recommendations.
 6. Streamlit displays the results.
-

Future Improvements

- Candidate profile upload
 - Resume parsing
 - Authentication
 - Conversation memory
 - Advanced filtering
 - Docker support
 - Cloud deployment
-

Author

Shivprakash Sahu

MCA (Artificial Intelligence & Machine Learning)

License

This project is developed for educational and demonstration purposes.